MindAI

Full Project Investigation Report

Architecture, Workflows, Data Model & Findings

Prepared for: jasserkhatib1@gmail.com

Repository: MindAi-main

Report date: 2026-07-02

Scope: nest-app (active backend) + client (Angular UI) + legacy src/ runner

Table of Contents

1. Executive Summary

MindAI is a conversational analytics product: a user asks a question in plain language, and the system turns that question into a MongoDB aggregation pipeline, executes it, and returns one of three product outputs - a Dashboard (ECharts widgets), a Report (structured prose sections), or an Inquiry (a single factual answer).

The repository currently contains two application generations living side by side:

- Active product: Angular 21 frontend in client/ talking to a NestJS 11 backend in nest-app/. This is the system actually used today.
- Legacy prototype: a standalone Express + Mastra (@mastra/core) TypeScript runner in the repository root (src/, server.ts, mastra/, http/, db/). It is kept for historical reference and is not part of the current runtime.

The active backend is built around a 'skills' architecture: a Planner skill converts natural language into a validated Mongo pipeline, a Pipeline service executes and validates it against declared source schemas, and output skills (Chart, Report Writer, Inquiry Writer, Memory extractor) transform the resulting rows into the final product payload. Large parts of the skill behavior live in Markdown prompt files under skills/*/SKILL.md rather than being hardcoded in TypeScript.

The system supports two alternative ways to reach an LLM: a user's personal API key, or a pool of shared 'agents' (saved provider/model/key connections) with health checks, automatic failover, and token-limit management. Persistence is split between MongoDB (sources, cache, history, saved results, settings, agent config, long-term memory) and a local LibSQL/SQLite file (short-term conversation threads and messages).

NOTE: This report describes the system as observed in the repository at the time of writing. Some backend behaviors (e.g. optimistic cron health checks) are noted as deliberate current trade-offs, not defects.

2. Repository Layout

Top-level layout of the repository (non-generated directories):

```
MindAi-main/
  client/            Angular 21 frontend (active)
    src/app/         Components + services (analytics-api, analytics-state, chart-render)
  nest-app/          NestJS 11 backend (active)
    src/analytics/   Controller, Service, PipelineService - core orchestration
    src/ai/          planner.ts, chart.ts, writer.ts, model.ts, memory-skill.ts, token.ts
    src/agent-config/  Shared agent CRUD + health-check cron
    src/sources/     Data source registry (schema metadata) + /api/meta
    src/user-settings/ Personal API key storage & validation
    src/memory/      Long-term extracted memory (Mongo)
    src/session/     Short-term conversation memory (LibSQL)
    src/cache/       Prompt-result cache
    src/history/     Pipeline run history
    src/saved-results/ User bookmarked results
    src/common/      Middleware, guards, filters, logger
  skills/            Markdown SKILL.md prompt definitions (aggregation, chart, report,
                     inquiry, memory, writer, suggestions, analytics)
  src/               LEGACY standalone Express + Mastra runner (not used by the active product)
  scripts/seed.ts    Legacy seed script
  data/              LibSQL database file(s) (memory.db)
  *.md / *.html      Prior documentation snapshots (ARCHITECTURE.md, HOW_IT_WORKS.md,
                     PROJECT_WORKFLOW_REPORT.md, README.md, and generated HTML guides)
```

3. Technology Stack

Active system

Layer	Technology
Frontend framework	Angular 21 (standalone components)
Frontend charts	Apache ECharts 5.5
Frontend styling	Tailwind CSS 3.4
Backend framework	NestJS 11 (Express platform)
AI SDK	Vercel AI SDK ("ai" 5.x) + provider adapters
LLM providers	OpenAI, Anthropic, Google Gemini, Groq, Mistral, Together AI, Perplexity
Agent framework (legacy pieces)	@mastra/core, @mastra/libsql, @mastra/memory
Primary database	MongoDB (via Mongoose)
Conversation memory store	LibSQL / SQLite (data/memory.db)
Validation	class-validator / class-transformer, Zod
Security middleware	helmet, custom ApiKeyGuard
Token estimation	gpt-tokenizer
Scheduling	@nestjs/schedule (cron)

Legacy runner (src/)

Layer	Technology
Runtime	Node.js + TypeScript (ESM)
HTTP server	Express
AI framework	Mastra (@mastra/core)
LLM provider	Groq (OpenAI-compatible API), llama-3.3-70b-versatile default
Structured output	Vercel AI SDK generateObject + Zod schemas
Database	MongoDB
Conversation memory	LibSQL via @mastra/libsql

4. Active System Architecture

Frontend

- Angular standalone app bootstraps from client/src/main.ts, mounting AppComponent.
- Main UI orchestration lives in client/src/app/app.component.ts.
- All API calls are centralized in client/src/app/analytics-api.service.ts.
- Local UI/session state lives in client/src/app/analytics-state.service.ts.
- ECharts rendering is done by client/src/app/chart-render.service.ts - the backend returns ready-to-render ECharts option objects; the frontend does not compute chart logic itself.

Backend

- Nest app entry point: nest-app/src/main.ts.
- Module wiring: nest-app/src/app.module.ts.
- Core execution path: analytics.controller.ts -> analytics.service.ts -> pipeline.service.ts.

Storage

MongoDB collections:

- sources - dataset schema metadata
- prompt_cache - cached aggregation/LLM results
- results_history - technical execution log per pipeline run
- saved_results - user-bookmarked outputs
- user_settings - personal API key + token settings
- agent_config - shared connection pool
- memory_items - long-term extracted memory

LibSQL/SQLite file data/memory.db stores short-term conversation threads and messages, managed by nest-app/src/session/memory.ts.

5. Backend Startup & Module Wiring

Boot sequence (nest-app/src/main.ts)

- Create the Nest application.
- Install AppLogger.
- Enable helmet.
- Enable CORS from ALLOWED_ORIGINS if defined, otherwise open CORS.
- Enable global validation via ValidationPipe.
- Start the HTTP server on PORT (default 3000).

Module composition (app.module.ts)

Imports: ConfigModule, ScheduleModule, MongooseModule, SourcesModule, HistoryModule, CacheModule, SavedResultsModule, UserSettingsModule, AnalyticsModule, AgentConfigModule.

Also installs RequestIdMiddleware (all routes), a global ApiKeyGuard, a global AllExceptionsFilter, and - if client/dist/mind-ui/browser/index.html exists - serves the built Angular app as static files, excluding /api and /health from the SPA catch-all route.

Health check

GET /health (app.controller.ts) pings MongoDB and checks that at least one source is registered; returns 503 if Mongo is unreachable or zero sources are loaded.

Request ID, logging, API key guard

- RequestIdMiddleware assigns a UUID to every request via AsyncLocalStorage.
- AppLogger prefixes logs with timestamp and request ID and supports colored tags.
- ApiKeyGuard: if API_KEY is not configured, requests are open; if configured, requests must send x-api-key. Public routes: /api/provider, /api/meta, /health.
- AllExceptionsFilter normalizes errors to { error, code?, ...extraFields }, preserving fields like currentLimit, suggestedLimit, agentApiKey that the frontend's token-warning UI depends on.

6. Main Request Lifecycle - POST /api/analytics

1. Controller entry

analytics.controller.ts validates prompt length, reads x-user-id, forwards to AnalyticsService.run().

2. Load configuration

analytics.service.ts loads personal user_settings and shared agent_config.

3. Resolve session & memory

Resolves/creates the session thread, loads short-term LibSQL context, loads relevant long-term Mongo memory snippets, combines both into LLM context (resolveSession(), buildMemoryContext()).

4. Resolve AI connection

resolveAccess(): a personal key wins if present; otherwise the backend picks an active agent, preferring one whose input limit fits the estimated request.

5. Input token preflight

estimateMinimumInputTokens() estimates prompt + memory-context tokens + fixed overhead; if it exceeds the connection's inputTokenLimit, raises INPUT_TOKEN_LIMIT_TOO_LOW before calling the LLM.

6. Execute by intent

executeByIntent() routes to PipelineService.executeDashboard() / executeReport() / executeInquiry().

7. Persist usage & metadata

Increments token usage on the agent or personal settings, updates lastInputTokens, returns result + token usage + connection info + any token warnings.

8. Persist conversation & memory

Saves the user+assistant turn to the LibSQL thread store; optionally runs memory extraction if enabled.

7. AI Execution Pipeline (Planner, Aggregation, Skills)

Core design

The backend uses prompt-driven AI skills rather than separate hardcoded backend services. The chain is:

- Planner skill: converts the prompt into a task plan + MongoDB aggregation pipeline.
- Mongo aggregation: executes the pipeline against the selected collection.
- Formatter skill: chart skill for dashboards, report writer for reports, inquiry writer for direct answers.
- Optional memory skill runs after the response is built.

Runtime prompt text is read from skills/*/SKILL.md via nest-app/src/ai/skill-prompt.ts - part of the system's behavior therefore lives in Markdown, not just TypeScript.

Planner workflow

runSupervisorPlan() (nest-app/src/ai/planner.ts) builds a structured plan containing: whether data is needed, the source name, the aggregation pipeline, a strategy, a chart hint, and whether a chart should accompany a report.

buildSchemaSection() enumerates every registered collection with exact fields, types, roles, enums and sample values; infers joins from referenceTo / description / name similarity; and adds copy-ready $lookup and pipeline templates. Planner quality is therefore directly dependent on source metadata quality.

The planner gets up to 2 attempts (pipeline.service.ts). The second attempt receives a corrective hint when the first plan fails field validation or Mongo execution, covering: multi-operator invalid stages, bad field names, projection inclusion/exclusion conflicts, incorrect date conversions, bad enum casing.

MongoDB aggregation workflow

- Data source resolution: plan.query.sourceName is resolved against the in-memory source registry (sources-cache.ts).
- Pipeline normalization: each stage must contain exactly one Mongo operator key; decoration keys are stripped with a warning.
- Field validation: referenced fields are validated against the declared source schema - this is the protection layer against planner field hallucination.
- Execution: db.collection(collection).aggregate(...) with allowDiskUse and maxTimeMS (env-configured, default 30s).

Empty-row behavior is intent-specific: Dashboard returns an empty dashboard state, Report returns a 'no data found' section, Inquiry may fall back to a generic could-not-be-answered summary.

8. Dashboard / Report / Inquiry Workflows

Dashboard

- Check full dashboard cache if there is no conversation context.
- Aggregate rows via planner + Mongo pipeline.
- If no chart skill selected, fall back to inquiry behavior.
- If zero rows, return an empty dashboard state.
- Otherwise call runChart(): asks the LLM for structured widget JSON, sanitizes the option object, injects dataset rows where needed, validates the renderable signal, supports table widgets, logs unknown encode references.
- Cache the full dashboard result.

Report

- Check full report cache if there is no context.
- Aggregate rows; if planner selects no report skill or rows are empty, return a 'No Data' section.
- Call runReportSkill() (writer.ts) using the report SKILL.md prompt - returns structured reportSections.
- If wantChart and enough rows exist, also call runChart().
- Merge report sections with the optional chart payload and cache the full report.

Inquiry

- Aggregate rows.
- If planner selected inquiry, call runInquirySkill() - takes the question + capped rows, returns one structured summary string.
- If no usable execution path remains, return: 'The request could not be answered from the available sources.'

9. Connections: Personal Keys & Shared Agents

Personal key

Stored in user_settings via user-settings.repository.ts: userId, apiKey, provider, model, responseTokenLimit, legacy-compatible inputTokenLimit, cumulative token usage. Validation (user-settings.service.ts) normalizes provider/model/key, validates provider existence, and performs a live provider API key validation request. Model dropdowns are a static curated list from nest-app/src/ai/model.ts, exposed via POST /api/settings/models - the dropdown no longer depends on a live provider fetch, but key validation still hits the provider.

Shared agents

An 'agent' here is a shared saved AI connection entry, not an autonomous AI worker. Each agent stores: provider, model, api key, input token limit, output token limit, cumulative input/output usage, last successful request input size, and status.

Status	Meaning
active	Eligible for selection
idle	Temporarily rate-limited
disabled	Manually turned off
expired	Invalid key, model failure, or quota exhaustion

Selection logic: personal key takes priority if present; otherwise the backend filters active agents, preferring one whose input limit fits the estimated request, else falling back to the first remaining active agent.

Save behavior (PUT /api/agent-config): saves the config document, immediately probes every non-disabled saved agent, and returns refreshed config - so statuses can change right after a save.

Runtime failover during a request: invalid key -> mark expired, try next; model not found -> mark expired, try next; provider rate-limited -> permanent exhaustion marks expired, temporary rate limit marks idle, then try next active agent. If no agent remains, the backend returns NO_ACTIVE_CONNECTION.

10. Agent Health Monitoring (Cron)

nest-app/src/agent-config/agent-health.service.ts runs every 5 minutes (@Cron('*/5 * * * *')). For each non-disabled agent it builds a provider-specific model-list or validation request, sends a probe, and marks the agent active (healthy) or expired (auth failure / permanently exhausted quota).

NOTE: The cron is intentionally optimistic in some cases: provider 404/500 does not expire the agent; model-not-in-list only logs a warning; temporary provider failures do not necessarily disable the agent. This means an agent card can show 'active' even though the next real generation call may still fail for provider-side reasons.

11. Token Management

Two independent dimensions

- Input token limit - how large the outgoing request to the model can be (mainly relevant to agent connections).
- Output/response token limit - how many tokens the model may generate back (relevant to both personal and agent connections).

Personal settings use responseTokenLimit as the primary field, mirrored to inputTokenLimit for backward compatibility. Each agent separately stores inputTokenLimit, outputTokenLimit, inputTokensUsed, outputTokensUsed, and lastInputTokens.

Input preflight

analytics.service.ts estimates the minimum request size from the current prompt + memory context + base overhead, compares it against the selected connection's input limit and against lastInputTokens from the previous successful call, and throws INPUT_TOKEN_LIMIT_TOO_LOW if it is clearly too small - avoiding a wasted round trip.

Output limit execution

The connection's output limit is sent as maxTokens/maxOutputTokens, then further clamped per stage by fallback caps: PLANNER_MAX_TOKENS, CHART_MAX_TOKENS, INQUIRY_MAX_TOKENS, REPORT_MAX_TOKENS. Effective allowance = connection output limit, then stage-specific cap if lower.

Post-call warnings & usage accounting

If output tokens reached the configured limit, the response includes tokenLimitExceeded and outputLimitWarning; if actual input tokens exceeded the input limit, it includes inputLimitWarning. A successful answer can therefore still return with a warning asking the user to raise limits. After a successful response, usage counters are incremented on the active connection (agent or personal), and memory-extraction tokens are charged back to that same connection.

12. Memory System (Short-term + Long-term)

Short-term conversation memory

nest-app/src/session/memory.ts stores sessions/threads/messages in LibSQL, keeping conversation continuity and powering the history sidebar and contextual follow-ups. Key functions: ensureThread(), getMemoryContext(), saveConversationTurn(), listSessions(), getSessionDetail(), deleteSession().

Long-term extracted memory

nest-app/src/memory/memory.service.ts retrieves relevant stored memory snippets before a request and optionally extracts new memory items after a response, via nest-app/src/ai/memory-skill.ts. Storage is MongoDB (memory.repository.ts).

MEMORY_EXTRACTION_ENABLED=false disables extraction at startup; the frontend can also toggle it at runtime via /api/memory/config. Even when extraction is disabled, existing memory items remain visible and retrievable.

13. Caching

nest-app/src/cache/cache.service.ts caches aggregation results by a normalized-prompt + intent hash, TTL 7 days, cache key SHA-256 truncated to 24 characters. Cache is used mainly when no conversation context is present, avoiding stale reuse for context-dependent follow-ups. PipelineService also maintains full-result caches for dashboard:full and report:full to skip repeated LLM work for identical context-free prompts.

14. History & Saved Results

- Pipeline run history (results-history.repository.ts): stores executed prompt, intent, Mongo collection, aggregation pipeline, rows, row count, duration - a technical execution log.
- Conversation history (session/memory.ts, LibSQL): stores user and assistant turns, powers the History tab in the UI.
- Saved results (saved-results.controller.ts): lets users bookmark results they want to keep; stored separately from raw history and scoped per user.

15. Error Handling & Recovery

Backend graceful recoveries

- Drops oldest memory context if the provider rejects the request due to context length.
- Retries the planner once with a corrective hint when a pipeline fails.
- Retries provider calls on short-term rate limits via withRateLimitRetry().
- Fails over across active agents at runtime.
- Returns friendly empty states for dashboard/report no-data scenarios instead of raw errors.

Backend hard failures

- Invalid personal API key.
- No active connection available (NO_ACTIVE_CONNECTION).
- Input limit too low (INPUT_TOKEN_LIMIT_TOO_LOW).
- Output limit too low (TOKEN_LIMIT_TOO_LOW).
- Unsupported structured-output model.

Frontend recoveries

- Refreshes agent config after failures.
- Opens the config panel automatically when no connection exists.
- Offers limit-raise actions inline.
- Shows a result footer with connection info so the user can see which key/model answered.

16. Frontend (Angular Client)

App initialization (AppComponent.ngOnInit)

- Create or reuse mind_user_id in browser localStorage.
- Load personal settings, dataset meta, session history, saved results, long-term memory items, memory extraction config, and agent config.

UI state (analytics-state.service.ts)

Tracks: current mode (dashboard/report/inquiry), prompt text, phase (idle/loading/done/error), current session ID, message thread, config sidebar state, saved results, memory items, agent config, pending report-format chooser, pending token-limit confirmation.

Modes and report special-case

Three user-facing modes map to backend intents: dashboard->dashboard, report->report, inquiry->general_question. In report mode, sending a prompt does not immediately call the backend - it first opens a local format picker (report only / chart only / both) via pendingSuggestion / chooseReportFormat(). This is a frontend product decision, not a backend requirement.

Frontend API surface (analytics-api.service.ts)

Method	Endpoint
GET	/api/meta
GET	/api/settings
POST	/api/settings/validate
POST	/api/settings
PATCH	/api/settings/token-limit
DELETE	/api/settings
GET	/api/agent-config
PUT	/api/agent-config
PATCH	/api/agent-config/token-limit
POST	/api/analytics
GET	/api/history/sessions
GET	/api/history/sessions/:id
DELETE	/api/history/sessions/:id
GET	/api/saved
POST	/api/saved
DELETE	/api/saved/:id
GET	/api/memory
DELETE	/api/memory
GET	/api/memory/config
PATCH	/api/memory/config

All user-scoped endpoints send an X-User-Id header.

Token UX

If the backend throws TOKEN_LIMIT_TOO_LOW or INPUT_TOKEN_LIMIT_TOO_LOW, the frontend builds a pending token confirmation card (buildTokenConfirm()). If a request succeeds but includes tokenLimitExceeded or inputLimitWarning, it uses buildPostCallConfirm(). Available user actions: Apply limit, Apply & Retry, Open Config, Dismiss, Keep partial response. Applying a limit calls PATCH /api/settings/token-limit (personal) or PATCH /api/agent-config/token-limit (agent), then retries for output-limit errors.

17. Data Model (MongoDB + LibSQL)

Collection / Store	Purpose
sources (Mongo)	Dataset schema metadata: fields, types, roles, enums, references
prompt_cache (Mongo)	Cached aggregation/LLM results, 7-day TTL
results_history (Mongo)	Every pipeline run: prompt, pipeline, rows, duration
saved_results (Mongo)	User-bookmarked outputs
user_settings (Mongo)	Personal API key, provider, model, token limits/usage
agent_config (Mongo)	Shared connection pool: provider, model, key, limits, usage, status
memory_items (Mongo)	Long-term extracted memory snippets
data/memory.db (LibSQL)	Short-term conversation threads and messages

18. Legacy Runner (src/) - Reference Only

NOTE: This layer is not used by the active product. It is documented here only so the history and prior design intent are not lost; do not mix it with the active Nest/Angular flow when explaining the current system.

MindAi's legacy root implementation is an Express + Mastra service that accepts a natural-language prompt and returns a Dashboard (full ECharts config), Report ({heading, body} sections), or Inquiry (single-sentence answer). The full pipeline runs in 2 LLM calls when intent is known, 3 when it is not.

Legacy data flow

```
User prompt -> POST /api/analytics { prompt, intent?, sessionId? }
  intent known?  -> executeDashboard / executeReport / executeInquiry
  intent unknown -> analyticsAgent.generate() (LLM routes to the right tool)
      -> runAggregation()
           1. runSupervisorPlan()   [LLM call #1]
           2. executePipeline()     [MongoDB]
           3. historyRepo.save()    [MongoDB, async]
      -> runChart() / runReport() / runInquiry()   [LLM call #2]
      -> saveConversationTurn()    [LibSQL]
      -> res.json(result)
```

Key legacy design decisions (still architecturally relevant)

- Full dataset schema is injected into the LLM system prompt every call - the model can only reference field names it has actually been shown, which keeps hallucination near zero.
- Sources are loaded once at startup into memory; no per-request DB round-trip for schema.
- Data-shape analysis runs in pure code before the chart LLM call, silently correcting impossible chart hints (e.g. scatter requested with only one numeric field).
- Chart rendering is pure code: the LLM only picks widget types and field names, never data values, keeping output deterministic and auditable. If none of the LLM's field names exist in the data, the renderer falls back to a table widget.
- results_history writes are fire-and-forget so a slow/failed write never delays the HTTP response.
- Direct tool dispatch when intent is known skips a full routing LLM round-trip.

The active nest-app backend clearly inherited this design philosophy (schema-in-prompt planning, pure-code chart rendering, fire-and-forget history) while re-platforming onto NestJS, adding multi-provider support, shared agent pooling, token governance, and long-term memory.

19. Known Behaviors, Risks & Recommendations

Known product behaviors

- Personal key always has priority over shared agents when a valid one is present.
- An agent shown as 'active' does not guarantee the next real request succeeds - cron health checks are optimistic in several failure modes.
- Skills (planner, report writer, inquiry writer, memory extractor) are partly prompt-based, so behavior can shift by editing skills/*/SKILL.md without a code deploy - this is powerful but also means prompt changes are a de-facto behavior change that should be reviewed like code.
- Source metadata quality directly controls AI planning quality; weak field descriptions/roles/enums degrade pipeline generation.
- Zero-row dashboard/report results are intentionally treated as friendly empty states, not failures.
- Token handling is both advisory and blocking - a valid response can still carry a warning asking the user to raise limits; this is expected, not a bug.

Observations worth follow-up

- Two parallel implementations of very similar logic (legacy src/ vs nest-app) increase maintenance surface; consider archiving/removing src/ once confidence is high nothing still depends on it.
- The optimistic cron health-check policy (Section 10) trades false 'active' status for fewer false 'expired' flaps - worth confirming this matches current reliability expectations as provider usage grows.
- Because planner/report/inquiry behavior is partly defined in Markdown (skills/*/SKILL.md), changes to those files should go through the same review/testing rigor as TypeScript changes, since they directly affect production output quality and safety (e.g. forbidden Mongo operators like $function/$merge/$out/$where/$eval should stay enforced both in the prompt and in code-level pipeline validation).
- Token accounting and failover logic is intricate (Section 11); consider dedicated tests around the edge cases (input limit too low pre-flight, output limit hit mid-stream, failover across expired/idle agents) if not already covered.

20. Recommended Mental Model

The cleanest way to explain the current system to a new team member:

- The frontend is a conversation-driven analytics UI, not a traditional dashboard builder.
- The Nest backend is the orchestrator, not just an API passthrough.
- The planner skill converts user language into a safe, schema-validated Mongo pipeline.
- MongoDB is the source of truth for both domain data and most application persistence.
- Writer/chart/memory skills transform raw rows into product-friendly outputs.
- Personal keys and shared agents are two alternative, interchangeable connection sources.
- Token limits, health probes, and failover are product-level controls wrapped around the LLM layer, not incidental plumbing.

Sources used for this report

This report synthesizes direct inspection of the repository (nest-app/src, client/src, skills/, root config files, package manifests) together with the project's own recent internal documentation (ARCHITECTURE.md and PROJECT_WORKFLOW_REPORT.md, both current as of this repository state), cross-checked against actual source files such as app.module.ts.